CS 24302 : COMPILER DESIGN LABORATORY

Lab Session 3 : Theme : Application of Pattern Recognition in Problem Solving

Scheduled : August 13 and 14, 2026

Faculty Team : Bikas Kanti Sarkar, Indrajit Mukherjee, Prashant Pranav, Mohd. Shah Fahad, Supreeti Kamilya and Supratim Biswas

Recap : 1. The s/w "lex" generates a C file named "lex.yy.c" and defines a function yylex(). The file contains the constructed DFA and definitions of variables and support functions.The function yylex() performs the role of pattern recognizer. It is essentially a driver function, that reads input, one character at at time, traverses the dfa in order to detect whether the input belongs to the language of dfa or not.
2. A single call to yylex() can detect all tokens in the input provided the actions associated with the regexes do not have a "return". Such use of yylex() is useful only during the development and testing phase of a scanner, but not for deployment.
3. In practice, a scanner is designed to return a single token to its caller. Such a scanner is implemented by inserting a "return" value at the end of each action and calling yylex() repeatedly till the EOF is reached, such as, "while (yylex());".
4. The order of regexes is important and it decides the final state that is reported by the recognizer.

Objectives :

- Given an application domain identify the lexical elements and write regular expressions.
- Acquire the skill to write regular expressions for an assorted collection of lexical elements, not only for programming languages, but also for other applications. Examples include, identifier, keywords, name of person, name of product, octal number, integer number, hexadecimal number, real number, string literals, character literals, operators, parentheses, delimiters, comments, date of event, timestamp, roll-number, employee-id, PAN, Aadhar, mobile, email-address, ip address, among many others.
- Create a working scanner for the tokens used in the chosen application. Write C codes in the actions part and other designated sections to solve the given problem.
- Experiment with the generated scanner and explore further extensions to solve larger problems.

P1. The application domain of this problem is Medical tests. Three common blood sugar tests are a) Fasting denoted as BS(F), b) Postprandial, usually 2 hours after lunch denoted by BS(PP), and c) HBA1C which is the average sugar score over the past 6 months. The range of values of each test for a normal adult is given below. Data about 5 patients are given in the format : i)date-of-test [dd-mm-yyyy], ii) name-of-patient [First alphabet in upper case, followed by lower case alphabets], iii) BS(F) with integer value, iv) BS(PP) with integer value, and v) HBA1C with a real value. The test scores are classified into 5 groups, denoted as Normal(N), Low(L), Very Low(VL), High (H) and Very High (VH).

Test Category	Test Name	Normal Range (N))	Low Range (L)	Very Low Range (VL)	High Range (H)	Very High Range (VH)
Blood Sugar	HBA1C	[3.8, 6.3]	[3.2, 3.8)	< 3.2	(6.3, 7.2]	> 7.2
	BS(F)	[66, 116]	[56, 66)	< 56	(116, 132]	> 132
	BS(PP)	[67, 147]	[56, 67)	< 56	(147, 168]	> 168

The notation (10, 29] stands for $10 < x \leq 29$, while [10, 29) stands for $10 \leq x < 29$, which are same as '(' and ']' for open and closed intervals on the real line.

(a) Given the sample input in the 1st column, create a scanner using Lex that produces the output given in 2nd column.

Input : "bstestinp1.txt"	**Desired Output :** "ver1a_out.txt"
01-01-2026 Anil BS(F) 48 BS(PP) 101 HBA1C 5.5 05-02-2026 Mita BS(F) 108 BS(PP) 150 HBA1C 6.4 11-03-2026 Sunil BS(F) 119 BS(PP) 170 HBA1C 6.1 29-04-2026 Seetha BS(F) 113 BS(PP) 140 HBA1C 6.5 07-08-2026 Shyam BS(F) 112 BS(PP) 131 HBA1C7.1	Patient No.1: 01-01-2026 Anil BS(F) 48 BS(PP) 101 HBA1C 5.5 || F:VL; PP:N; HBA1C:N Patient No.2: 05-02-2026 Mita BS(F) 108 BS(PP) 150 HBA1C 6.4 || F:N; PP:H; HBA1C:H Patient No.3: 11-03-2026 Sunil BS(F) 119 BS(PP) 170 HBA1C 6.1 ||F: H;PP:VH; HBA1C:N Patient No.4: 29-04-2026 Seetha BS(F) 113 BS(PP) 140 HBA1C 6.5 ||F:N; PP:N; HBA1C: H Patient No.5: 10-08-2026 Shyam BS(F) 112 BS(PP) 131 HBA1C 7.1 || F:N; PP:H; HBA1C :H

(b) In this version, the input fields can appear in any order, while keeping the test name and its score together. Design a scanner for processing such test record inputs and output the patient records in exactly the same order as that of part(a) above. (Hint: you may use a table with the fields in the predefined order and construct the table as a line of input is processed).

Input :"bsrandtinp1.txt"	**Desired Output :** "ver1b_out.txt"
BS(F) 48 01-01-2026 Anil BS(PP) 101 HBA1C 5.5 05-02-2026 BS(PP) 150 BS(F) 108 HBA1C 6.4 Mita Sunil HBA1C 6.1 BS(PP) 170 BS(F) 119 11-03-2026 HBA1C 6.5 BS(F) 113 BS(PP) 140 29-04-2026 Seetha BS(PP) 131 HBA1C 7.1 10-08-2026 Shyam BS(F) 112	Patient Test Date Name BS(F) BS(PP) HBA1C F PP HBA1C 1 01-01-2026 Anil 48 101 5.5 VL N N 2 05-02-2026 Mita 108 150 6.4 N H H 3 11-03-2026 Sunil 119 170 6.1 H VH N 4 29-04-2026 Seetha 113 140 6.5 N N H 5 10-08-2026 Shyam 112 131 7.1 N N H

Resources given : pathological-tests.pdf "bstestinp1.txt" "ver1a_out.txt" "bsrandtinp1.txt" "ver1b_out.txt"

P2. Consider the C program in the file, "sampleprog.c". This program has no errors and produces some output when compiled and executed (verify for yourself). Browse the source to observe the tokens and their structure. You task is to create a scanner that can recognize tokens in the C program and generate an output as given in the file, "sampleout.txt". The derived output for the first few lines of the C source is given below.

#include <stdio.h>
#define x 10
int main()
{ // sample C program for testing lexical analyser
int i = 0x40;
……..
	pre-processor directive ignored : #include <stdio.h>
	pre-processor directive ignored : #define x 10
	(Token, Lexeme) = (KEYWORD, int)
	(Token, Lexeme) = (IDENTIFIER, main)
	(Token, Lexeme) = (LEFTPAR, '(')
	(Token, Lexeme) = (RIGHTPAR, ')')
	(Token, Lexeme) = (LEFTPAR, '{')
	Single Line Comment = "// sample C program for testing lexical analyser"
	(Token, Lexeme) = (KEYWORD, int)
	(Token, Lexeme) = (IDENTIFIER, i)
	(Token, Lexeme) = (ASSIGN-OP, '=')
	(Token, Lexeme) = (HEXA NUMBER, 0x40)
	(Token, Lexeme) = (DELIMITER, ';')
	…...

Resources given : "sampleprog.c" "sampleout.txt"

P3. Applications that examine structured information content, such as logfiles, to detect some unusual patterns that indicate presence of possible malicious intent, are domains that can benefit from this technology. Consider the directory data structure in Linux file system, and the command "ls -asl" that accesses the kernel file system data structure to display several attributes of files and directory resources, $ ls -asl > lsout.txt. Explanation of the symbols used in the display.

type : single letter, "d" denotes a directory and "-" a regular file
permissions : 3 letters from the list {r,w,x,-}, one triple each for {owner, group, others}, total of 9 letters.
"r", "w" and "-" denote read access, write access and no access respectively.
"x" for a file indicates it is executable, for a directory indicates permission to search it.
Hard link count (hlcnt): Number of time a file / dir is listed in the file system.
Assume there are no shared files or directories in the file system,
hlcnt(f) = 1 where f is a file; and hlcnt(d) = 2 + number-of-child-directories(d), for a directory d.
Illustration of a few sample entries are given below.

# disk blocks	Type & access permissions	Hard Link count times listed in f.s.	Owner id	Group id	Size In bytes	Time last modified : month date & time	Name of file / dir
4	drwxrwxr-x	6	sb	sb	4096	Aug 11 15:13	• ≡ curr dir
4	drwxrwxr-x						•• ≡ parent dir
72	-rw-rw-r--	1	sb	sb	72805	Aug 8 19:30	filename_rules.pdf

Design a scanner to process the output displayed by "ls – asl" command to answer the following questions . You may save the contents of your home directory on the server, in a file and use it for this problem.
a) How many executable files are there in the directory?
b) List the file / dir with the largest and smallest size.
c) Which resources are not accessible to other users?
c) List the oldest and the most recent resource.

End of Lab Session 3